Kunai: Toward a Verifier-Safe Layered DSL for Nested Encapsulation Packet Filtering on eBPF

Anonymous Author(s)

ABSTRACT

Packet capture and network debugging require selecting the packets of interest early, inside the Linux kernel. However, pcap-filter, tcpdump's filter language, has no syntax for nested encapsulation, repeated occurrences of the same protocol, or the variable-length lists and options whose field offsets change from packet to packet. Such encapsulations are common in production traffic. Running a filter in the kernel additionally requires emitting bytecode that the Linux eBPF verifier accepts, and a naive lowering of such filters is rejected. We present Kunai, a packet-filter DSL together with a compiler that lowers its expressions to eBPF bytecode. A Kunai expression describes a packet's layer structure and field conditions together, and resolves each field's offset and type from protocol definitions written in a subset of P4. The compiler records every header's start offset at run time, bounds each variable-length scan by a compile-time constant, and places a verifier-compliant bounds check before every packet read. We evaluate Kunai on filters spanning GTP-U, SRv6, Geneve, and TCP options. The generated bytecode is accepted by the verifier across six kernels (Linux 6.1–7.0) on both the XDP and tc hosts, matches and rejects the intended packets, and runs within a few percent of pcap-filter per packet. Kunai enables the Linux kernel to execute filters that pcap-filter cannot express.

KEYWORDS

eBPF, XDP, packet filtering, domain-specific language, P4, kernel verifier, packet capture, network debugging

1 INTRODUCTION

Packet capture is a basic operation in debugging and fault analysis of network systems. As link bandwidth and traffic volume grow, forwarding every packet to user space for storage no longer scales, so a capture tool must select only the packets relevant to its purpose early, inside the kernel. The de-facto language for this selection has been tcpdump/libpcap's pcap-filter [17, 26].

pcap-filter expresses conditions on L2–L4 header fields. It has, however, no syntax for nested encapsulation, repeated occurrences of the same protocol, or lists and options whose length varies from packet to packet. In the inner IPv4 of a GTP-U or Geneve tunnel, an SRv6 segment list, or TCP options, the offset of the target field changes per packet [1, 8, 9, 11]. These encapsulations are common rather than exceptional: GTP-U carries mobile user-plane traffic, SRv6 and Geneve underlie datacenter and WAN overlays, and TCP options such as MSS and window scaling appear on the SYN of nearly every connection, so a kernel capture tool encounters them. Filter languages that can follow these structures by field name exist (the Wireshark display filter [28] is one), but they evaluate a fully parsed packet held in user-space memory, where any field can be read by random access; the kernel data path offers no such parsed view and requires every read to be proven in bounds, so these languages cannot run early in the data path. User studies of packet-analysis tools likewise report protocol coverage and the expressiveness of the filter language as open problems [25].

Expressiveness alone is not enough. To run such a filter inside the kernel, it must also be compiled to eBPF bytecode that the verifier accepts [6, 10, 16]. Compilers from P4 to eBPF, such as p4c-xdp [27], already emit verifier-accepted code, but they lower fixed protocol pipelines rather than filter expressions whose target field sits past nested encapsulation or along a variable-length list. For such a field the offset is not known until run time, and a naive lowering reads at a position the verifier cannot prove in bounds, so the bytecode is rejected. To our knowledge, no system offers both field-name expressiveness over these structures and a form the in-kernel verifier accepts.

We present Kunai, a filter DSL and compiler that meets both requirements. Kunai describes a packet's layer structure as a *layer-chain* and resolves the offset and type of each field from protocol definitions written in a subset of P4 [19], then lowers the filter expression to eBPF bytecode. It records each header's start offset at run time, bounds every variable-length scan by a compile-time constant, and places a verifier-compliant bounds check before every packet read, so the verifier accepts the generated code; we describe the code generation in §4. We use the term *verifier-safe* in the sense of Solleza et al. [24] (a well-formed filter should compile to bytecode the verifier accepts) and establish it empirically across six kernels (§5), not as a formal guarantee (§6).

We show that filters spanning GTP-U and SRv6 are expressible in Kunai, that the generated bytecode is accepted by the verifier across six kernels (Linux 6.1–7.0) on both the XDP and tc hosts, that it matches and rejects packets as intended, and that its per-packet cost stays within a few percent of pcap-filter's where both can express it.

2 RELATED WORK

tcpdump/libpcap's pcap-filter descends from the Packet Filter [18] and the BSD Packet Filter [17] and runs L2–L4 conditions in the kernel. As §1 noted, it has no syntax for repeated protocols or for walking variable-length lists such as an SRv6 segment list or TCP options by field name; the more expressive Wireshark display filter [28] runs only in userspace. Tools that capture at the XDP layer do not close this gap. xdpcap filters in the kernel but with a pcap-filter [4], inheriting the same limit; xdpdump has no in-kernel filter, capturing via fentry/fexit and leaving matching to tcpdump [21]. Neither selects packets in the kernel by a field condition past nested encapsulation.

eBPF'26, September 29, 2026, Prague, Czech Republic
2026. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Prior work on encapsulation-aware filters includes NetPDL/NetPFL and pFSA/xpFSA. NetPDL/NetPFL connect a description of protocol headers with tunnel-aware filter expressions [3, 22]. pFSA/xpFSA represent the order in which headers appear within an encapsulation as state transitions and formalize the composition of filters and field access [2, 14]. These share our goal of bringing nested encapsulation into the filter expression, but do not emit eBPF bytecode for the Linux kernel data path, so lack the in-kernel execution this paper requires.

Among combinations of P4 and eBPF, p4c-ebpf and p4c-xdp compile a P4 data-plane program to Linux eBPF / XDP / tc [20, 27], and Honey for the Ice Bear embeds dynamically loaded eBPF inside a P4 pipeline [23]. These move an entire P4 pipeline to the target rather than returning per-packet match/reject from a filter expression; Kunai instead uses P4 only as a protocol definition, not as a data-plane language [19].

We adopt the recent *verifier-safe* goal argued by Solleza et al. [24]: a kernel-extension DSL should compile every well-formed program to bytecode the verifier accepts. Kunai is a concrete instance of that position for packet filtering over nested encapsulation: rather than restricting an existing DSL, we identify the code-generation patterns the goal demands when a filter reaches past variable-length structure (§4).

The work above has separately addressed packet-filter execution, filter expression over nested encapsulation, P4-to-eBPF compilation, and the goal of verifier-safe lowering. Kunai joins two of these: it brings nested encapsulation and variable-length structure into the filter expression and compiles the result into bytecode the verifier accepts, generalized across the protocols declared in the P4 subset (§4).

3 DESIGN

3.1 Overview

Figure 1 shows the overall structure of Kunai. The Kunai compiler takes two inputs. The first is a Kunai DSL expression, the filter condition written by the user. The second is a set of protocol definitions written in a subset of P4, a database of protocol headers and parser rules.

The Kunai compiler first interprets the layer-chain: it checks that each protocol may connect to the next according to the definitions and, when the same protocol appears more than once, binds a label to each layer slot. It then resolves field references against the protocol definitions, obtaining the layer, offset, and bit width of each field. When a reference targets an auxiliary header list or an option, the scan target and the information needed to walk it are also recorded in the IR. This analysis stage rejects three things: a reference to a non-existent field; an unqualified `proto.field` reference, such as `tcp.dport`, when that protocol appears more than once; and a comparison whose type or bit width does not match.

The output of the Kunai compiler is eBPF bytecode that evaluates the filter condition against a packet and returns a match/reject result. At run time, a host BPF program placed at an attach point such as XDP [12], tc [13], or fentry/fexit embeds this bytecode. Under this split of responsibilities, the Kunai compiler handles filter-condition analysis and bytecode generation, while the host BPF program handles attach-point-specific context access and follow-on actions such as capture or drop.

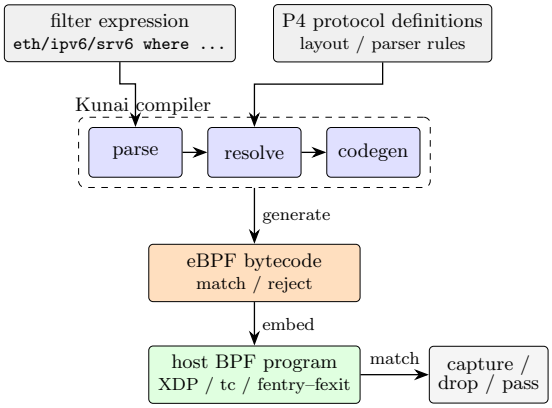

Figure 1: Overview of Kunai. From a filter expression and P4 protocol definitions, the compiler produces match/reject eBPF bytecode through parse/resolve/codegen, which a host BPF program embeds at an attach point.

```
filter ::= layer-chain where-clause?
layer-chain ::= layer ('/' layer)*
layer ::= layer-atom quantifier?
layer-atom ::= proto-name ('@' label)? predicate?
             | '(' layer ('|' layer)+ ')'
quantifier ::= '?' | '+' | '*' | '{' INT '}' | '{' INT ',' INT '}'
predicate ::= '[' comparison (',' comparison)* ']'
where-clause ::= 'where' bool-expr
bool-expr ::= comparison | quant-atom
            | '(' bool-expr ')' | bool-expr bool-op bool-expr
comparison ::= field-ref op value
quant-atom ::= ('any' | 'all') '(' bool-expr ')'
field-ref ::= ident index? ('.' ident index?)*
```

Figure 2: Excerpt of the Kunai filter-expression grammar.

3.2 What the DSL Expresses

The Kunai DSL describes a packet's layer structure and field conditions as a single filter expression; Figure 2 shows an excerpt of the grammar; the full grammar is available in the project repository.[1]

A layer-chain lists, with `/`, the order in which protocols appear in the packet, and can also express optional layers, repetition, and choice. The `where` clause is a condition over the fields of layers in the chain and of auxiliary headers and lists. Kunai resolves a field reference into one of the forms in Table 1; there, `<stack>` is not a literal but the name of an auxiliary header list given in the protocol definition, such as `segments` or `exts`.

Reference form (example)	Meaning
`proto.field` (`tcp.dport`)	primary field of a protocol unique in the chain
`label.field` (`inner.dst`)	field of a layer carrying a label (`@inner`)
`proto.aux.field`	field of an optional or auxiliary header
`proto.<stack>[N].field`	static index into an auxiliary header list
`proto.<stack>.field`	element being walked in `any()`/`all()`
`proto.options.NAME.field`	TCP / IPv4 option lookup

Table 1: Field-reference forms Kunai resolves.

[1] https://github.com/(redactedforreview)

Kunai interprets a field reference inside a bracket predicate as relative to that layer. In the `where` clause, references name the layer explicitly, as in `proto.field` or `label.field`.

When the same protocol appears more than once in a chain, `proto.field` alone does not determine which layer to read. For example, when a chain contains both an outer and an inner IPv4 header, `ipv4.dst` does not uniquely identify which destination field is meant. Kunai rejects such a reference before code generation and requires a label-qualified reference. A filter on the inner IPv4 destination of a GTP-U tunnel, for example, is written as follows.

```
eth/ipv4@outer/udp/gtp/ipv4@inner/tcp where inner.dst ==
    10.0.0.1
```

The chain before `where` lists the layers to traverse, from Ethernet through an outer IPv4 header, UDP, GTP, an inner IPv4 header, to TCP, and the `where` clause gives the field condition checked once the chain matches, here that the inner IPv4 header's destination equals `10.0.0.1`. Here `@outer` and `@inner` distinguish the two IPv4 layers. Because `inner.dst` uniquely names the destination field of the inner IPv4 header, Kunai can determine which field to read even though the same `ipv4` header appears twice.

Variable-length lists are handled with `any()` and `all()`. A filter that matches a packet whose SRv6 segment list contains a given address, for example, is written as follows.

```
eth/ipv6/srv6 where any(srv6.segments.addr == fc00::1)
```

The scan target of `any()` and `all()` is the auxiliary header list referenced without an index inside the expression, and there must be exactly one such target list. The same list may be referenced several times; for example, `any(srv6.segments.addr == fc00::1 or srv6.segments.addr == fc00::2)` is valid. An `any()` or `all()` that references no un-indexed list, or two different ones, is rejected at the analysis stage rather than walking one of them, so a quantifier that appears to walk two lists cannot be written. Kunai evaluates the inner condition against each element of the list. The walk requires a field giving the element count, the width of each element, a way to advance to the next element, and a stop condition; the protocol definition supplies these. TCP options and GTP extension headers are handled the same way when the protocol definition carries this information.

Through these constructs, the Kunai DSL specifies, within one filter expression, the order in which headers appear, the disambiguation of repeated protocols, and the target of a variable-length walk. Code generation uses this information to translate each field access into an offset from the layer's start and a bit width.

3.3 Protocol Definitions via a P4 Subset

Kunai uses a subset of the P4 language to describe protocol headers and parser rules [19]. Because P4 already has syntax for protocol headers and parser transitions, existing P4 parser descriptions can serve as a reference when writing a protocol definition. Using P4 also avoids implementing and maintaining the syntax checker, parser, tooling, and specification that a bespoke protocol-description language would require.

The P4 subset Kunai handles is limited to the parts that describe header layout and parser transitions. Figure 3 shows the SRv6 protocol definition; the `any()` example in §3.2 and filter F8 in §5 use it.

```
header srv6_h {
    bit<8> next_header;
    bit<8> hdr_ext_len; // in 8-byte units
    bit<8> routing_type; // 4 = SRH
    bit<8> segments_left;
    bit<8> last_entry;
    bit<8> flags;
    bit<16> tag;
}
header srv6_seg_h { bit<128> addr; }
const bit<8> SRV6_IPV6_NEXT_HEADER = 43;
parser SRv6Parser(packet_in pkt, out srv6_h hdr,
    @kunai_layout[after=primary]
    @kunai_stack_count[field=last_entry, offset=1]
    out srv6_seg_h[8] segments) {
  state start {
    pkt.extract(hdr);
    transition select(hdr.routing_type) {
      4: skip_segments; default: reject;
    }
  }
  state skip_segments {
    pkt.advance(((bit<32>)(hdr.hdr_ext_len & 0x0F)) << 6);
    transition accept;
  }
}
```

Figure 3: SRv6 protocol definition in the Kunai P4 subset.

This definition supplies the information the Kunai compiler needs through three constructs. First, the `header` declarations give the offset and bit width of each field. Second, the `const` and `transition select` give the dispatch from IPv6 to SRv6 and the acceptance test on `routing_type`. Third, the annotations give the walk metadata for the variable-length part: `@kunai_layout` indicates that the `segments` list begins immediately after the fixed header fields of the protocol, and `@kunai_stack_count` that the element count is read as `last_entry + 1`; the bounded walk of §4 uses these. The capacity 8 of `srv6_seg_h[8]` bounds the iteration count, and the masked `pkt.advance` in `skip_segments` caps the variable-length advance at 120 bytes, both exposing static bounds to the verifier.

In this way, a protocol definition describes layout, dispatch, and walk metadata within P4 syntax. Adding a protocol definition extends the set of supported protocols without changing the compiler itself.

4 VERIFIER-CONSTRAINED CODE GENERATION

This section describes how Kunai lowers a filter expression into the packet accesses, loops, and bounds checks that the Linux eBPF verifier accepts. When nested encapsulation or a variable-length structure makes a subsequent header's start offset vary per packet, reading a field at that run-time offset leaves its range unprovable to the verifier, so the bytecode is rejected. The code generation below avoids it.

The difficulty is not detecting that an offset is dynamic but lowering the read, the same way for every layer past a variable-length one

and for every protocol in the P4 subset, into a form the verifier accepts. Kunai's code generation does this by recording each changing header's start offset at run time (§4.1) and walking variable-length structures such as SRv6 segments and TCP options under a compile-time bound (§4.2), with a verifier-compliant bounds check before every packet read.

4.1 Recording Header Start Offsets Explicitly

Kunai records each header's start offset at run time and reads a field as a relative offset from that start. Consider again the GTP-U inner-IPv4 filter of §3.2: `inner.dst` is not at a fixed offset, since the start of the inner IPv4 changes with the outer IPv4 length, the GTP optional fields, and the structure of the encapsulation.

A fixed compile-time offset does not suffice: if the variable-length parts before the field (GTP optional fields or IPv4 options) are non-empty, it reads the wrong bytes. The offset must be computed at run time. But a run-time offset against the native-XDP packet pointer (`PTR_TO_PACKET`) is one the verifier cannot bound, so it cannot prove the read stays within `data_end` and rejects the bytecode.

Kunai detects this boundary at the resolve stage and marks every layer after one with a variable-length layout as a run-time offset. The generated eBPF bytecode then records each header's start offset into a stack slot as it traverses the header at run time. Figure 4 illustrates this offset recording and a bounds-checked load that uses the recorded offset.

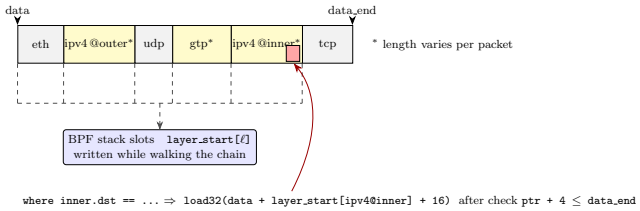

Figure 4: Recording each header's start offset into a stack slot, and reading `inner.dst` as a bounds-checked load at layer-start + field offset.

Every field access takes the same form: Kunai loads the layer's start offset, adds the field offset and width, checks the access against `data_end`, loads, and compares. Because type-mismatched comparisons and a `proto.field` reference under a repeated protocol are already rejected at the analysis stage of §3.1, code generation need only emit this bounds-checked load, even for a chain whose subsequent-header start changes at run time.

4.2 Lowering Variable-Length Structures to Bounded Bytecode

In variable-length structures such as SRv6 segments and TCP options, the element count or the length of each element changes per packet, so they cannot be walked at a fixed offset. Kunai puts the walk into a verifier-accepted form by translating it into bytecode with a compile-time constant bound.

Consider again the SRv6 segment-list filter of §3.2. Because the SRv6 segment list varies in element count per packet, the filter must walk the list in order rather than compare at a fixed offset.

Kunai generates the walk as one of two kinds of bytecode, chosen by the bound and applied uniformly to chain quantifiers and `any()`/`all()` aux-list walks. A small compile-time bound within a fixed threshold is unrolled in place, which is compact when the bound is small. A larger or unbounded one is lowered to a `bpf_loop` callback with a constant bound [7], whose size no longer grows with the bound; this case covers a parser-machine self-loop, an unbounded chain quantifier, and a chain or aux-list walk past the threshold. An SRv6 segment walk and a large MPLS stack take the same lowering. For SRv6 segments, the `@kunai_stack_count` annotation in the listing of §3.3 gives the actual element count as `last_entry + 1`, so each iteration checks that it does not exceed that count.

Algorithm 1 shows this walk; the code generator instantiates it per protocol.

Algorithm 1 Bounded element walk.

Require: *data*, *data_end*, *base*, *stride*, *width*, *count*, *cap*, *target*
1: $matched \leftarrow$ false; $off \leftarrow base$
2: **for** $i \leftarrow 0$ **to** $cap - 1$ **do** ▷ *cap*: compile-time bound
3: **if** $i \geq count$ **then break** ▷ stop past the element count
4: $ptr \leftarrow data + off$
5: **if** $ptr + width > data\_end$ **then break** ▷ bounds check
6: **if** load(ptr) = *target* **then** $matched \leftarrow$ true; **break**
7: $off \leftarrow off + stride$ ▷ *stride* constant, or read per element
8: **return** *matched*

For the SRv6 filter above, `base = srh_start + SRH_FIXED_SIZE`, `stride = width = 16`, `count = last_entry + 1`, and `target = fc00::1`, with `cap` the compile-time bound.

Kunai generates the TCP-options walk with the same structure. Because TCP options differ in length per option and a reference such as `tcp.options.MSS.value` cannot be located at a fixed offset, `stride` is not a fixed value but the length of each option, read as the cursor `off` advances. In place of the element-count test, the walk stops when the cursor reaches the end of the options, when an END option is read, or when the iteration bound is reached. It also stops on a malformed option, such as `len < 2` or `cursor + len > options_end`. An option-specific field is loaded only after checking that its option header does not cross the end of the packet window.

The Kunai compiler treats the bound of a variable-length walk as a constant determined by the protocol definition and the code-generation settings. SRv6 segments and TCP options differ in structure, but Kunai generates both as bounded walks and, in either the fixed-unrolling or the `bpf_loop` form, places a bounds check before each packet load. In this form, the verifier can confirm both the loop bound and the range of every packet access.

So that the generated bytecode is accepted across kernel versions, Kunai avoids instructions available only on newer versions. For byte-order conversion, for example, it uses the `BPF_END` instruction, available on older versions, rather than the `BSWAP` instruction introduced in 6.6 [15].

These two patterns are not specific to Kunai but are what a verifier-safe lowering of nested, variable-length filters requires. *Recorded-base addressing* reads each field as a recorded layer-start

base plus a static offset, spilling the base to a stack slot during traversal, which makes an otherwise unprovable run-time read provable. *Constant-bounded scanning* bounds every variable-length walk by a compile-time cap, unrolled when small and lowered to a `bpf_loop` callback when large, so both the trip count and each read are provable. Only the base, run-time or compile-time constant, varies between reads; the read shape is uniform, so one generator covers GTP-U, SRv6, Geneve, and TCP options without per-protocol cases.

5 EVALUATION

We evaluate Kunai from three angles. First, whether the packet structures Kunai targets can be written as filter expressions (§5.2). Second, whether the generated bytecode is accepted by the eBPF verifier and matches/rejects as expected (§5.3). Third, the instruction count and run-time cost of the generated bytecode (§5.4).

5.1 Method and Filter Set

The evaluation uses ten filters, F1–F10, listed in Table 2. F1–F6 target basic protocol headers such as Ethernet, VLAN, IPv4/IPv6, TCP, and ICMP, while F7–F10 target nested headers and variable-length structures whose start cannot be fixed at a constant offset. It lists each filter's Kunai expression and the raw eBPF instruction count of the bytecode Kunai generates, together with the count for the same filter compiled from pcap-filter to eBPF through cBPF [17] using cbpfc [5], a cBPF-to-eBPF compiler. We refer to this latter count as the pcap-filter baseline in the rest of the evaluation.

To exercise the DSL syntax more broadly, we hand-constructed a separate regression suite of 77 filter expressions, at least one per syntactic feature of the DSL. The suite covers bracket predicates, arithmetic in the `where` clause, quantified layers, heterogeneous-size alternations, label disambiguation, and aux-walks, spanning a wider syntactic range than F1–F10. Its verifier acceptance and per-packet match/reject correctness are reported in §5.3.

To evaluate these filters, we embedded the generated bytecode in a host BPF program and placed it at the XDP and tc attach points. The remaining subsections use this filter set and regression suite throughout.

5.2 Expressiveness

We compared whether F1–F10 can be written in pcap-filter, the filter language that runs in the kernel, and in Kunai. Because pcap-filter has byte-offset access, the one expressiveness axis Kunai does not follow (§6), a hand-written condition can sometimes be built for a specific packet layout. It does not, however, provide as filter syntax the order of headers, an arbitrary element of a variable-length list, repeated occurrences of the same protocol, or a named reference to an inner header.

Result: F1–F6 were writable in both; only F5 (QinQ) behaved in pcap-filter in a way that depended on the capture environment, since NIC VLAN offload can strip the tag from the bytes. The two diverge at F7–F10. The inner IPv4 header of F7 (GTP-U) and F9 (Geneve) sits at no fixed offset, shifted per packet by GTP optional fields and extension headers or by a Geneve inner Ethernet header; F8 (SRv6) requires looping over an arbitrary segment-list element; and F10 (TCP MSS) requires an option walk rather than a byte-offset approximation. None of these is expressible as pcap-filter syntax, whereas Kunai writes all of F1–F10 directly through the labeled layer-chains, `any()`/`all()` quantifiers, and option-field walks listed in Table 2.

5.3 Verifier Acceptance and Correctness

We made two checks of the code generation of §4: first, that the generated bytecode is accepted by the eBPF verifier; second, that the accepted bytecode matches and rejects as expected.

For acceptance, we loaded the bytecode generated from F1–F10 and the regression suite onto six kernels (Linux 6.1, 6.6, 6.12, 6.15, 6.18, 7.0) at both the XDP and tc attach points. The instructions and helpers the verifier accepts differ across kernel versions, but because Kunai avoids version-dependent instructions as described in §4, the same bytecode was accepted unchanged on all six versions.

At tc, the kernel extracts the outer VLAN tag into skb metadata before the program runs, so Kunai rejects at compile time a filter that reads a VLAN/QinQ field from packet bytes. An optional, predicate-free tag stays matchable: at most one tag remains in the bytes, so `eth/vlan?/...` takes its skip path and matches untagged, single-tagged, and QinQ frames without reading the stripped tag. This is a compile-time restriction, not a verifier rejection. We excluded from the tc loads only the field-reading forms, 2 of F1–F10 and 3 of the regression suite, and performed 1014 loads across six kernels × 2 hosts, all accepted.

Next, we fed the generated bytecode actual packets (matching, non-matching, and malformed) and checked the match/reject result. For Geneve we varied `opt_len` from 0 upward to confirm the inner offset resolves past the skipped options, with truncated and malformed frames throughout, and the bytecode returned the expected verdict in every case. The regression suite is verified at two levels: all 77 expressions load on the verifier, and 74 of them are also checked for match and reject correctness at the packet level; the 3 capture-clause expressions are load-only, since their verdict equals the base chain. For the five filters both languages express (F1 to F4 and F6) we also cross-checked Kunai against libpcap, the engine behind tcpdump, on the same packets, and the verdicts agreed throughout, an independent reference beyond the hand-built expectations.

Result: These two checks show that the code generation of §4 lowers the target filters into bytecode accepted across six kernels × 2 hosts and traverses packet headers correctly.

5.4 Instruction Count and Run-Time Cost

We first evaluate a filter's cost by the instruction count of the generated eBPF bytecode, listed per filter in Table 2. The instruction count is the static size of the bytecode and is comparable regardless of the measurement environment; a 64-bit immediate load is counted as two instructions for both Kunai and pcap-filter. On the F1–F4 and F6 filters that pcap-filter can also express, Kunai generated 2.7× to 14.7× as many instructions, the largest gap arising at the IPv6 prefix match of F3.

Because every variable-length walk is bounded at compile time, a filter's instruction count grows with that bound, not with the packet: a chain quantifier adds about 16 instructions per unit of cap while it unrolls, then flattens once it lowers to a `bpf_loop` callback, and each stacked SRv6 segment walk, itself such a callback, adds

ID	Kunai expression	raw insns	
		Kunai	pcap-filter
F1	`eth/ipv4/tcp where tcp.dport == 443`	199	42
F2	`eth/ipv4[src==10.0.0.0/8]/tcp where tcp.dport == 80`	209	36
F3	`eth/ipv6[src==2001:db8::/32]/tcp`	338	23
F4	`eth/vlan[tci==100]/ipv4/tcp where tcp.dport == 80`	215	51
F5	`eth/qinq/vlan/ipv4/tcp where tcp.dport == 80`	217	—
F6	`eth/ipv4/icmp where icmp.type == 8`	85	31
F7	`eth/ipv4@outer/udp/gtp/ipv4@inner/tcp where inner.dst == 10.0.0.1`	405	—
F8	`eth/ipv6/srv6 where any(srv6.segments.addr == fc00::1)`	316	—
F9	`eth/ipv4@outer/udp/geneve/eth/ipv4@inner/tcp where inner.dst == 10.0.0.1`	351	—
F10	`eth/ipv4/tcp where tcp.options.MSS.value == 1460`	231	—

Table 2: Evaluation filter set and raw eBPF instruction counts. F7–F10 target nested headers and variable-length structures. The pcap-filter column is the eBPF compiled from each filter via cBPF; — marks those pcap-filter cannot express.

a fixed ≈70. Even eight stacked walks stay near 800 instructions, three orders of magnitude below the verifier's roughly one-million-instruction limit.

To see whether this appears at run time, we connected a traffic generator to a device under test (DUT) over 100 GbE and attached a native XDP program that evaluates the filter and returns `XDP_DROP` on every packet. The DUT runs Linux 6.15 with the BPF JIT on an Intel Xeon 8362 and an E810 NIC. As in an operational fentry/tc observer, the program first copies the packet head into a per-CPU scratch window; this copy alone lowered the rate by 6.7% even with no filter, so we measured each filter's cost as the processing-rate reduction relative to that same-copy accept-all baseline.

Figure 5 shows the per-filter processing-rate reduction. We measured at the 64-byte line rate and repeated each cell ten times; every standard deviation was at most 0.4.

Result: The reduction for the basic filters F1–F6 was 0–1.7%, near 0% for F6. The Kunai-only filters F7–F10 were 3.9–6.6%, with F7, which entails a full chain walk, the largest at 6.6%. For F1, writable in both Kunai and pcap-filter, the difference between the two was only 0.7%.

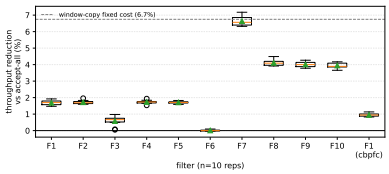

Figure 5: Datapath processing-rate reduction for F1–F10 (accept-all baseline, box plot, n=10). The most complex filter F7 (6.6%) is within the 6.7% window-copy cost (dashed); triangles are means.

A per-packet run-time measurement via `BPF_PROG_TEST_RUN` agreed: over a filterless accept-all baseline of 611–612 ns, each filter added at most +15 ns/pkt (under 3%), and the Kunai–pcap-filter difference was at most the measurement standard deviation. The cost is therefore dominated by the fixed per-packet work every program on the datapath shares, not by filter evaluation.

6 LIMITATIONS

Kunai is a DSL specialized for packet filtering, not a general-purpose eBPF language: stateful filtering, deep packet inspection, and aggregation are out of scope, and P4 defines only header layout and parser transitions, not pipeline mechanisms such as actions and tables.

Kunai addresses every field by name and offers no escape hatch for a raw byte offset, as pcap-filter's `ip[14:2]` does. This exclusion is deliberate: only a named, schema-resolved access lets the generator bound each read into a form the verifier accepts, which an arbitrary run-time offset does not. The raw byte-offset hatch is thus a one-directional gap by design, not a shortfall. Sub-byte fields such as a SYN-flag test are still written by name as `where tcp.flags & 0x02 != 0`, with the generator masking a byte-aligned covering window.

Our verifier-acceptance result is empirical, not a formal guarantee: we confirmed acceptance and matching on six kernels for F1–F10 and the regression suite, but acceptance for an arbitrary Kunai expression on an arbitrary kernel version is future work. Because a variable-length walk is translated into bounded bytecode, the element count it can walk has a compile-time bound, and a filter that uses `bpf_loop` is subject to a kernel-version lower bound.

The generated bytecode is only as correct as the protocol definition: the p4c parse check verifies P4 syntax, not that field offsets or parser transitions match the relevant RFC. The definitions also have coverage limits: Geneve option TLVs, for example, are not yet exposed as filter targets. Finally, Kunai's bytecode is a host-agnostic evaluator over a packet window, and each host adapter supplies the layout it sees; at tc, where the kernel moves the outer VLAN tag into skb metadata, reading that tag value is future work (§5.3).

7 CONCLUSION

We presented Kunai, a DSL and compiler that makes packet-filter conditions over nested encapsulation, repeated protocols, and variable-length lists and options executable inside the Linux kernel. Across six kernels (Linux 6.1–7.0) and both the XDP and tc hosts, the generated bytecode is accepted unchanged, matches and rejects as intended, and adds at most 15 ns per packet, within measurement noise of pcap-filter where both can express the filter.

REFERENCES

[1] 3GPP. 2023. TS 29.281: GPRS Tunnelling Protocol User Plane (GTPv1-U). 3GPP Technical Specification.

[2] Ivano Cerrato and Fulvio Risso. 2018. Enabling Precise Traffic Filtering based on Protocol Encapsulation Rules. *Computer Networks* (2018).

[3] Luigi Ciminiera, Mario Baldi, Riccardo Sisto, and Fulvio Risso. 2010. Tunnel-aware Filter Languages: NetPFL. In *Proc. IEEE GLOBECOM*.

[4] Cloudflare. 2019. xdpcap: XDP Packet Capture. Blog post / GitHub: cloudflare/xdpcap.

[5] Cloudflare. 2021. cbpfc: a cBPF-to-eBPF compiler. https://github.com/cloudflare/cbpfc.

[6] eBPF Docs. 2024. Loops in BPF. https://docs.ebpf.io/linux/concepts/loops/.

[7] eBPF Docs. 2024. `bpf_loop` helper function. https://docs.ebpf.io/linux/helper-function/bpf_loop/.

[8] W. Eddy. 2022. Transmission Control Protocol (TCP). RFC 9293.

[9] C. Filsfils et al. 2020. IPv6 Segment Routing Header (SRH). RFC 8754.

[10] Elazar Gershuni et al. 2019. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. In *Proc. ACM PLDI*.

[11] J. Gross, I. Ganga, and T. Sridhar. 2020. Geneve: Generic Network Virtualization Encapsulation. RFC 8926.

[12] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. 2018. The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel. In *Proc. ACM CoNEXT*.

[13] iproute2 project. [n. d.]. tc(8): show / manipulate traffic control settings. Linux man page.

[14] Marco Leogrande, Fulvio Risso, and Luigi Ciminiera. 2015. Modeling Complex Packet Filters with Finite State Automata. *IEEE/ACM Transactions on Networking* (2015).

[15] Linux Kernel Documentation. 2024. BPF Instruction Set Architecture (ISA). https://docs.kernel.org/bpf/standardization/instruction-set.html.

[16] Linux Kernel Documentation. 2024. eBPF Verifier. https://www.kernel.org/doc/html/latest/bpf/verifier.html.

[17] Steven McCanne and Van Jacobson. 1993. The BSD Packet Filter: A New Architecture for User-level Packet Capture. In *Proc. USENIX Winter Technical Conference*.

[18] Jeffrey C. Mogul, Richard F. Rashid, and Michael J. Accetta. 1987. The Packet Filter: An Efficient Mechanism for User-level Network Code. In *Proc. ACM SOSP*.

[19] P4 Language Consortium. 2023. P4-16 Language Specification. https://p4.org/p4-spec/docs/P4-16-v1.2.5.pdf.

[20] P4 Language Consortium. 2024. P4C Documentation: eBPF Backend. https://p4lang.github.io/p4c/ebpf_backend.html.

[21] The XDP Project. [n. d.]. xdpdump (xdp-tools). https://github.com/xdp-project/xdp-tools.

[22] Fulvio Risso and Mario Baldi. 2006. NetPDL: An Extensible XML-based Language for Packet Header Description. *Computer Networks* (2006).

[23] Manuel Simon, Henning Stubbe, Sebastian Gallenmüller, and Georg Carle. 2024. Honey for the Ice Bear – Dynamic eBPF in P4. In *Proc. ACM SIGCOMM Workshop on eBPF and Kernel Extensions (eBPF'24)*. https://doi.org/10.1145/3672197.3673436

[24] Franco Solleza, Justus Adam, Akshay Narayan, Malte Schwarzkopf, Andrew Crotty, and Nesime Tatbul. 2025. Kernel Extension DSLs Should Be Verifier-Safe!. In *Proc. ACM SIGCOMM Workshop on eBPF and Kernel Extensions (eBPF'25)*. https://doi.org/10.1145/3748355.3748368

[25] Noa Sultana et al. 2024. A Survey on Packet Filtering. *ACM SIGCOMM Computer Communication Review* (2024).

[26] tcpdump.org. 2024. pcap-filter(7) manual page. https://man7.org/linux/man-pages/man7/pcap-filter.7.html.

[27] William Tu et al. 2018. p4c-xdp: Compiling P4 to XDP. In *Linux Plumbers Conference*.

[28] Wireshark Foundation. 2024. Wireshark Display Filter Reference. https://www.wireshark.org/docs/dfref/.